继承先建立子对象，再提供基类视图

C++17 • 类定义

```
struct Base {
  int id = 7;
  const char* kind() const { return "Base"; }
  virtual const char* speak() const {
    return "Base";
  }
  virtual ~Base() = default;
};
struct Derived : public Base {
  int score = 90;
  const char* kind() const { return "Derived"; }
  const char* speak() const override {
    return "Derived";
  }
};
```

main 内：引用和指针绑定同一个对象

```
Derived d;
Base& r = d;
Base* p = &d;
```

类关系：派生类指向基类



d 的逻辑子对象结构



r.score 编译错误：成员查找使用 Base 接口。

虚调用选择实现，按值复制改变对象

main 内：沿用上一页的 Base / Derived

```
Derived d;
Base& r = d;
Base* p = &d;
r.kind();
r.speak();
p->speak();
r.Base::speak();
Base copy = d;
copy.speak();
```

按值复制会切片

d：Derived 对象
Base 部分 + score

按值复制 →

copy：Base
id = 7

override 核对覆盖签名；
参数或 const 不匹配会触发编译诊断。

表达式的返回值与选择依据

表达式	返回	选择依据
r.kind()	Base	非虚，按 Base 类型
r.speak()	Derived	虚调用，动态类型是 Derived
p->speak()	Derived	同一个 d，经指针调用
r.Base::speak()	Base	显式限定，调用 Base 实现
copy.speak()	Base	copy 是独立 Base 对象

成员查找先检查静态类型的接口。
未显式限定的虚调用，再选择动态类型的最终覆盖者。

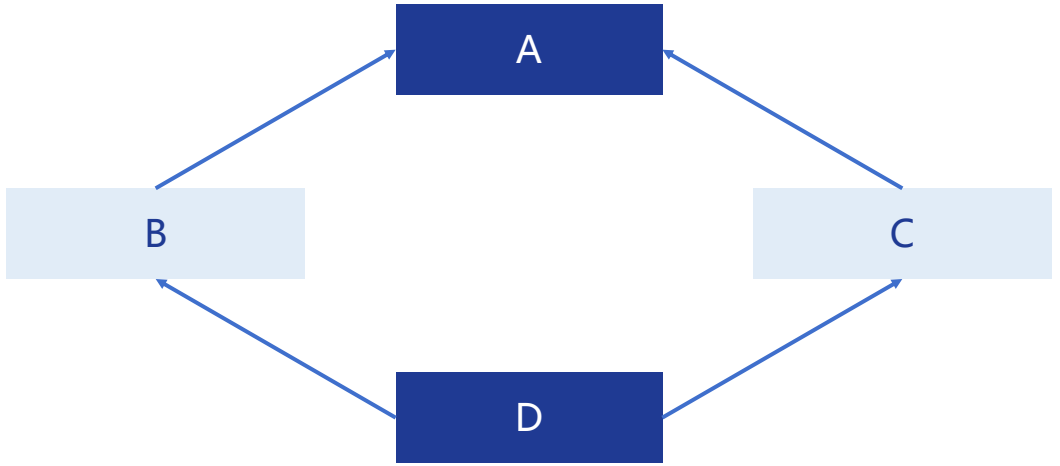
经 Base* 删除动态创建的 Derived 时，
虚析构使派生析构链执行。

普通菱形：两条路径保留两份 A

C++17 • 类定义与构造日志（包含 <iostream>）

```
struct A {
  int id;
  explicit A(int n) : id(n) {
    std::cout << "A(" << id << ") ";
  }
};
struct B : A {
  B() : A(1) { std::cout << "B "; }
};
struct C : A {
  C() : A(2) { std::cout << "C "; }
};
struct D : B, C {
  D() { std::cout << "D\n"; }
};
```

继承关系：箭头由派生类指向基类

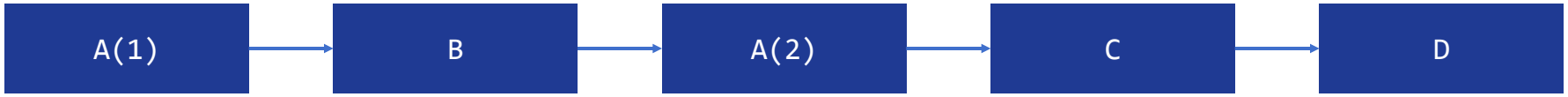


按声明顺序构造 B、C；各自先构造自己的 A。

B::A：id = 1　　**C::A：id = 2**

编译错误：A& a = d;
原因：从 D 到 A 的基类转换有歧义。

D d; 的输出



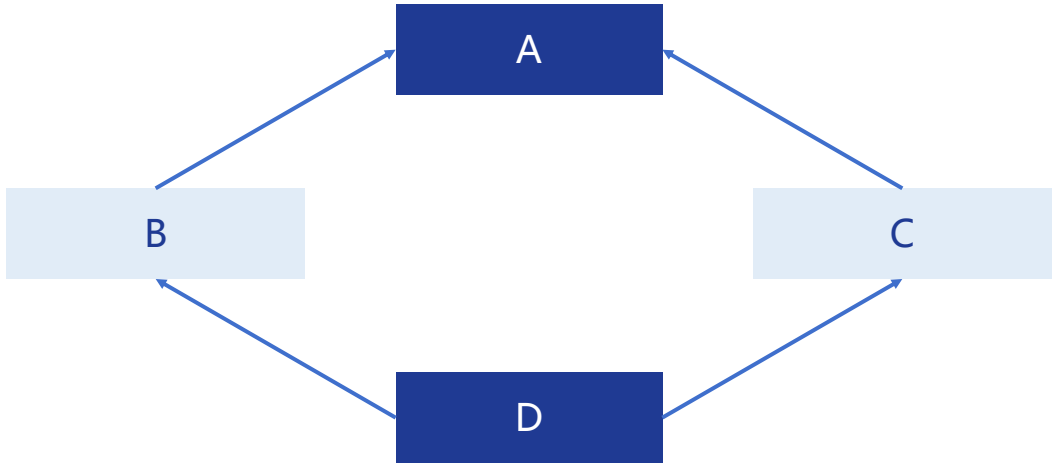
虚继承：共享 A，由最派生类初始化

C++17 • 类定义与构造日志（包含 <iostream>）

```
struct A {
  int id;
  explicit A(int n) : id(n) {
    std::cout << "A(" << id << ") ";
  }
};
struct B : virtual A {
  B() : A(1) { std::cout << "B "; }
};
struct C : virtual A {
  C() : A(2) { std::cout << "C "; }
};
struct D : B, C {
  D() : A(9) { std::cout << "D\n"; }
};
```

D 构造时，B/C 的 A 初始化器被忽略。
单独构造 B：A(1) 生效。

继承关系：箭头由派生类指向基类



先构造虚基类 A，再按声明顺序构造 B、C。

共享 A：id = 9；经 B、C 转成 A* 后指针相等

D d; 的输出



共享 A 的虚函数需要唯一最终覆盖者

C++17 • A 提供虚函数，D 统一覆盖

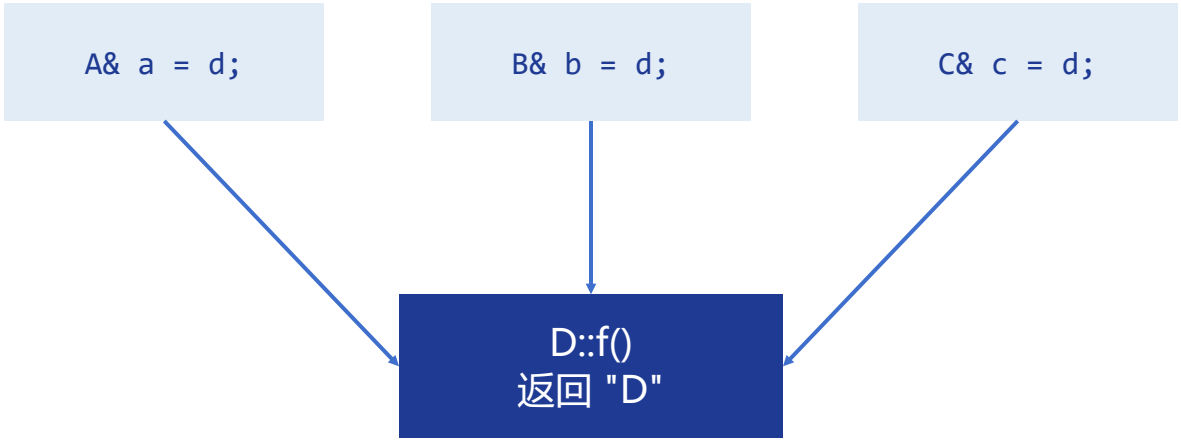
```
struct A {
  virtual const char* f() const { return "A"; }
  virtual ~A() = default;
};
struct B : virtual A {
  const char* f() const override { return "B"; }
};
struct C : virtual A {
  const char* f() const override { return "C"; }
};
struct D : B, C {
  const char* f() const override { return "D"; }
};
```

显式限定可直接调用某一路实现

```
D d;
d.B::f();  // returns "B"
```

先辨认“有几份基类子对象”，再检查“虚函数最终由谁覆盖”。

同一个 D 对象，三种基类引用



若 D 没有覆盖 f，B::f 与 C::f 同时成为候选；
共享 A 的最终覆盖者不唯一，类定义编译失败。

virtual 函数 选择调用实现	virtual 继承 共享基类子对象